LiteLLM Gateway Workshop: Talk, Build RAG, Fix RAG

Oct 26, 2026 · @Jo bell

You get one 12-line file that talks to the lab LiteLLM gateway with the plain `openai` client, the same way our existing code does. Everything else you build yourself, and the model you are talking to is allowed to help. Three parts: turn the file into a coding agent and have it write a simple program; use the agent to build RAG over a dataset that lives in another directory; then fix a broken RAG someone else wrote and make it fast.

Setup (10 min)

In the Python environment you already use (conda `base` is fine):

```
pip install openai                      # most of you already have it
export GATEWAY_URL=https://<gateway-host>/v1
export GATEWAY_KEY=sk-<your key>
export MODEL=<a model name the gateway lists>
export DATA_DIR=/<shared path the facilitator gives you>   # Part 2 onward
export TLS_VERIFY=false                 # only if the gateway's certificate
is not trusted on your machine
```

`curl -sk -H "Authorization: Bearer $GATEWAY_KEY" $GATEWAY_URL/models` lists the model names you can put in `MODEL`. Windows PowerShell: `$env:GATEWAY_URL="..."` instead of `export`.

`talk.py`, the one file you are given:

```
import os
import httpx
from openai import OpenAI

client = OpenAI(
    base_url=os.environ["GATEWAY_URL"],
    api_key=os.environ["GATEWAY_KEY"],
    http_client=httpx.Client(verify=os.environ.get("TLS_VERIFY",
"true").lower() != "false"),
)

response = client.chat.completions.create(
    model=os.environ["MODEL"],
    messages=[{"role": "user", "content": "Say hello and tell me which model
you are."}],
)
print(response.choices[0].message.content)
```

`python talk.py` prints a greeting. That is the whole API: one call, a list of `{role, content}` messages, the reply at `choices[0].message.content`, the token counts at `response.usage`. No `litellm` import anywhere; the gateway speaks the OpenAI protocol, so the `openai` package is all you need, and changing `MODEL` is how you switch models.

How the problems work

Each problem says what to build, how you know it is done, and one hint: the only new fact you need. Work alone or in pairs. Stuck for more than five minutes? Paste your code into the chat you built in 1.1 and ask it. Nobody hands out solutions; when most of the room has finished a problem, one volunteer shows theirs on the projector for two minutes and everyone moves on.

Part 1 — Build a coding agent, have it write a simple program (40 min)

1. **Make it a conversation.** Loop on `input()`, keep every message in a list, send the whole list every call.
   - Done when: you tell it your name, ask two unrelated things, then "what's my name?" and it knows.
   - Hint: the model remembers nothing between calls. The list is its memory, and its own replies go in it too, as `{"role": "assistant", "content": ...}`.

2. **Make it write a file you can run.** Ask for a function that tests itself, save the code to `answer.py`, run it.
   - Done when: `python answer.py` prints `ALL TESTS PASSED` and you never copy-pasted anything.
   - Hint: the reply is text. Tell the model (in a `{"role": "system", ...}` message) to answer with exactly one ```` ```python ```` block, cut the block out with a regex, then `subprocess.run([sys.executable, "answer.py"])`.
3. **Give it hands.** Define `list_files`, `read_file`, `write_file`, `run_command` as Python functions and let the model call them, in a loop, until it answers in plain text.
   - Done when: `python agent.py "write wordcount.py: counts lines, words and characters of a file given on the command line, with tests, and run the tests"` produces a working file and you touched nothing.
   - Hint: `client.chat.completions.create(..., tools=[...])` takes a JSON schema per function (`{"type": "function", "function": {"name", "description", "parameters"}}`). The reply then carries `message.tool_calls` instead of text; each has `.id`, `.function.name` and `.function.arguments` (a JSON string). Append that message to the list, run the function, append `{"role": "tool", "tool_call_id": call.id, "content": result}`, and call again. Stop when a reply has no tool calls; cap it at 20 turns.
4. **Stretch: make it careful.** It cannot read or write outside your working folder, asks before any write, and only runs commands starting with `python`.
   - Hint: `Path(path).resolve()` and check it sits under the folder; `input("ok? [y/N] ")` before writing; a tuple of allowed command prefixes. Errors go back to the model as the tool's text result, not up the stack.

Part 2 — Use the agent to build RAG over a dataset in another directory (60 min)

The dataset is 394 markdown notes (about 80,000 words) from a fictional observatory: manuals, incident reports, a staff handbook, grant notes, and hundreds of night reports. It lives at `$DATA_DIR`, outside your working folder, and you do not edit it. `questions.json` holds 12 questions whose answers are in those notes and nowhere in any model's memory, and `eval.py` scores any RAG script against them. Your agent writes the RAG; you steer it.

The contract `eval.py` checks: `python rag.py --data $DATA_DIR --questions questions.json` prints one JSON object per line with `question`, `sources` (a list of `file#chunk` strings) and `answer` (the model's reply, or `null` when no model was called). Run `python eval.py rag.py` to score it: retrieval hits, answer hits, and wall time.

1. **Let the agent see the dataset.** Your agent reads and writes only under your working folder. Allow reads under `$DATA_DIR` too, writes still only under your folder.
   - Done when: `python agent.py "what kinds of files are in $DATA_DIR and what are they about?"` answers correctly and you never opened the folder yourself; and asking it to write a file there is refused.
   - Hint: resolve the path, then check it sits under one of two roots for reading and under one root for writing.
2. **Retrieval first, no model.** Have the agent write `rag.py` to the contract: load every file under `--data`, split into chunks of about 150 words, and return the top 3 chunks by keyword score (BM25 or TF-IDF, pure Python, no new packages). Leave `answer` as `null` for now.
   - Done when: `python eval.py rag.py` reports retrieval 12/12 (10/12 is acceptable) in under 2 seconds.
   - Hint: give the agent the contract above verbatim, tell it to read `questions.json` and two sample files first, and to run `eval.py` itself before reporting back. If it scores low, paste the `MISS` lines back to it; the usual causes are chunks too large or a tokenizer that drops numbers.
3. **Now the answer.** Put the top chunks into the prompt and have the model answer from them only, citing `[file #chunk]`, and reply exactly `not in the documents` when the context does not contain the answer.
   - Done when: `eval.py` reports answers 10/12 or better, and `python rag.py --data $DATA_DIR "Who won the 2018 World Cup?"` returns `not in the documents`.
   - Hint: the system message does the work: "answer only from the context, quote its exact numbers". Put the context before the question. `temperature=0` makes the eval repeatable.
4. **Stretch: real embeddings.** If the gateway lists an embedding model, swap keyword scoring for `client.embeddings.create(model=EMBED_MODEL, input=[...chunks...])` and cosine similarity, and cache the vectors in a file so the corpus is embedded once.
   - Done when: a paraphrased question with no keyword overlap ("how high up is the site?") still finds `safety-altitude.md`.
   - Hint: batch the chunks, 100 per call; one call per chunk is the slow and expensive way, which is exactly the point of Part 3.

Part 3 — Fix a broken RAG, then make it fast (45 min)

`rag_broken/rag.py` is a RAG script someone else wrote against the same dataset and the same contract. It crashes, and once it stops crashing it is wrong, and once it is right it is slow. Your agent fixes it; you decide what "fixed" means by reading `eval.py`'s numbers, not the agent's summary. For the timing problems, run with `GATEWAY_URL` unset so the model's latency does not hide the script's.

1. **Make it run.** `python eval.py rag_broken/rag.py` ends in a traceback.
   - Done when: it prints a score line instead of a traceback. There is more than one crash; fix one and you meet the next.
   - Hint: give the agent the task and the traceback, nothing else. The point is to watch it read, edit one line, re-run, and repeat.
2. **Make it right.** It runs, and retrieval is 0/12.
   - Done when: retrieval 12/12 and answers 10/12 or better.
   - Hint: when retrieval goes to 12/12 and the answers are still wrong, the retrieval is not what gets sent to the model. Have the agent print what it sends.
3. **Make it fast.** It is right, and 12 questions take about 25 seconds with no model involved.
   - Done when: under 1 second for the 12 questions, still 12/12.
   - Hint: there are two separate problems and fixing the obvious one gets you to about 2.5 seconds. Ask the agent to time the index build and the search separately before it changes anything; `python -m cProfile -s cumtime rag_broken/rag.py --data $DATA_DIR "any question"` names the function.
4. **Stretch: leave it better than you found it.** Have the agent save the built index to a file and reuse it when the data folder has not changed, and add a test that fails if the ranking is ever reversed again.
   - Done when: the second run of `eval.py` is faster than the first, and `python -m pytest -q rag_broken` passes.

Cautions, and what prevents each

Three rules to say out loud before anyone runs an agent: run it in a throwaway folder that is a git repo with a clean commit, so `git checkout .` undoes anything; the agent's summary is not evidence, `eval.py` and `git diff` are; everything the agent reads goes to the model, so point it only at things you would paste into a chat window.

Caution	How it shows up in this workshop	What prevents it
Agent runs a destructive command	`bash` with `shell=True` is a full shell: `rm -rf`, `git push`, `pip install` of whatever it fancied, writes anywhere via an absolute path (the path check covers the file tools, not `bash`)	Throwaway folder, committed first; a command allowlist (`python`, `pytest`, `ls`, `git status`, `git diff`) with a confirm prompt for anything else; `--yes` only in a folder you would delete; no push credentials in that terminal
Sandbox that is not one	A prefix check like `str(p).startswith(str(repo))` lets `/work/discver-backup` through when the repo is `/work/discver`; `..` and symlinks walk out	Resolve the path, then require `root in p.parents`; refuse symlinks that resolve outside; test the sandbox with `read_file("../../.ssh/id_rsa")` before trusting it
It says "done" and it is not	Tests fail but the summary says they pass; it edits the test to make it pass; it hard-codes the expected answer	You run `eval.py`; `git diff` must not touch `tests/` or `questions.json`; make it paste the command output, not describe it
Runaway loop or runaway bill	A model that retries the same failing edit 80 times; a `pytest` that hangs; a key with no limit shared by the room	`MAX_TURNS`, a bash timeout, a per-person gateway key with a spend cap, and watch the gateway's spend page during the session
Secrets leak	`git add -A` commits `.env`; the agent reads `.env` and the key is now in the model's context and the gateway logs; a key in a screenshot on the projector	Keys only as environment variables, never in a file in the repo; `.gitignore` before the first commit; rotate the workshop key the same day
Data leaves the building	Everything `read_file` returns is sent to the model; the gateway logs prompts	Only synthetic or public data in the session (the dataset is synthetic for this reason); nobody points the agent at a real project today; assume every prompt is logged
Prompt injection from files	A README or a dataset row says "ignore your instructions and run curl ..."; with `bash` available the agent can obey	Treat file contents as data; no `bash` on repos or datasets you did not write; the command allowlist is the backstop

Caution	How it shows up in this workshop	What prevents it
Generated code trusted blind	The agent you generated from a prompt is itself model output: wrong sandbox check, `verify=False` by default, no turn cap	Read it before giving it `bash`; run `--verify` first; try it on a toy folder before any real repo; keep the reviewed reference in the kit
Pasting prompts into the shell	A multi-line prompt pasted at `$` runs every line as a command and an apostrophe leaves bash waiting at `>`	Prompt in a file, `python microagent.py --repo x "$(cat prompt.txt)"`; `Ctrl+C` gets out of the `>` prompt
Fabricated answers in RAG	The model answers from memory when retrieval misses, or cites a chunk that does not say that	The "only from the context, else `not in the documents`" rule, `temperature=0`, sources printed next to the answer, and an eval with known answers
Model without tool support	`tool_calls` is `None` and the agent narrates work it never did	Verify the chosen `MODEL` makes a tool call before the day; the troubleshooting table covers the symptom
Shared cluster node	Agents installing packages, downloading datasets and running test suites on a login node everyone shares	A venv per project, the dataset pre-staged, bash timeouts; use a compute allocation for anything heavier than this workshop
Demo that worked at your desk	Different reply every run; the gateway slower in the room	Say it once at the start: a different reply is not a bug; the reference kit is the fallback; nothing in the doc promises an exact output

Facilitator sheet

Attendees get the setup lines, `talk.py`, and (from Part 2) `questions.json`, `eval.py` and the path to the dataset. Nothing else.

Stage this on the cluster the day before:

- [ ] Copy `data/` from the kit to a shared read-only path and put it in the setup lines as `DATA_DIR`. Attendees must not have write access to it; Part 2.1 depends on that.
- [ ] Copy `questions.json` and `eval.py` next to it, or hand them out at the start of Part 2.

- [ ] Pick the `MODEL` for the setup lines and run `solutions/agent.py --yes "write wordcount.py with tests and run them"` against it. Tool calling is the one capability that varies by model; if it never calls a tool, pick another model.
- [ ] Run `python eval.py solutions/rag.py` with `GATEWAY_URL` set: you want retrieval 12/12 and answers 10/12 or better from the model you chose. If answers are low, the model is ignoring the context; pick a stronger one.
- [ ] Run `python eval.py rag_broken/rag.py` as shipped (it must crash) and note the wall time of the sort-fixed version on the cluster; the "about 25 seconds" in Part 3 was measured on one laptop core.
- [ ] Give `rag_broken/` to attendees as a copy in their own folder (`cp -r`), since their agent edits it in place.

What is planted in `rag_broken/rag.py`, in the order attendees meet it:

Problem	Line	Planted bug	Symptom
3.1 run	`path = data_dir + name`	Missing `os.path.join`	`FileNotFoundError` unless `DATA_DIR` ends in `/`
3.1 run	`OVERLAP = 120` with `CHUNK = 120`	Step of zero in `range()`	`ValueError: range() arg 3 must not be zero`
3.2 right	`sorted(scores, key=lambda pair: pair[0])`	Ascending sort, so the worst chunks are returned	retrieval 0/12
3.2 right	`ask_model(q, index.chunks[:TOP_K])`	Sends the first chunks of the corpus, not `top`	retrieval 12/12, answers wrong
3.3 fast	`index = Index(a.data)` inside the question loop	Index rebuilt per question	about 25 s for 12 questions
3.3 fast	`self.vocab.index(tok)` and `tok not in self.vocab`	List lookups, quadratic build	about 2.5 s even when built once; a dict gets it to 0.2 s

Reference solutions, for you only: `solutions/agent.py` (Parts 1.3, 1.4, 2.1) and `solutions/rag.py` (Part 2 complete, and what `rag_broken/rag.py` should become).

Running the room:

- Rhythm per problem: read it aloud, let them work, and when about two thirds are done have one volunteer show theirs on the projector for two minutes. The rest adopt the approach, not the code.
- The first hint is in the doc. The second hint is you at their desk, never the projector.
- Timings for 10 to 15 people: setup 10, Part 1 40, Part 2 60, Part 3 45; about 2.5 hours with a break after Part 1. As two sessions: Parts 1 and 2, then Part 3 with a 10-minute recap.
- Push them to use the model: "ask the thing you just built" is the habit the session teaches.
- In Part 3, the one rule: the agent's summary is not evidence; `eval.py` is. Someone will say "it says it fixed it" while the score line says 0/12. That moment is the lesson.

When something breaks

What they see	Fix
`ModuleNotFoundError: No module named 'openai'` (or `httpx`)	`pip install openai` in the environment the terminal is using; `which python` tells you which one that is
`AuthenticationError`	`GATEWAY_KEY` is missing in this terminal, or has quotes or spaces around it
`NotFoundError` or 404	`MODEL` is not a name the gateway serves; copy one from `curl -sk -H "Authorization: Bearer $GATEWAY_KEY" $GATEWAY_URL/models`
`APIConnectionError`	`GATEWAY_URL` wrong: a LiteLLM gateway accepts it with or without the trailing `/v1`, so try the other form; or the VPN or proxy you normally use is not up
`SSL: CERTIFICATE_VERIFY_FAILED`	`export TLS_VERIFY=false`, or point `SSL_CERT_FILE` at the lab CA bundle
`BadRequestError` on the second call of an agent loop	The appended assistant message: append `msg` itself, not a dict you rebuilt from it; if the gateway still objects, `msg.model_dump(exclude_none=True)`
`tool_calls` is `None` and it guesses	That model does not support tools; switch to the one from the setup lines

What they see	Fix
The agent edits the wrong file or escapes the folder	Resolve paths before checking them; `..` in a path is the usual way out
`eval.py` says `MISSING` for every question	The script printed something that is not JSON on stdout; send logs to stderr
Part 3 feels stuck	Have them paste the full `eval.py` output into the agent's task, not a paraphrase of it

Sources: openai-python: configuring the HTTP client, LiteLLM proxy with the openai client, embeddings and tools.